

AI4012 — Assignment #1 Report

Student: 23i-2578 | **Task:** #23 Cybersecurity / Vulnerability — CVE intelligence (GET /v1/security/cve)

Section 1 — System Architecture

The system answers one question for an AI agent: “*What is the current, trusted information about this CVE?*” It is a Python 3 / FastAPI / PostgreSQL pipeline with four moving parts.

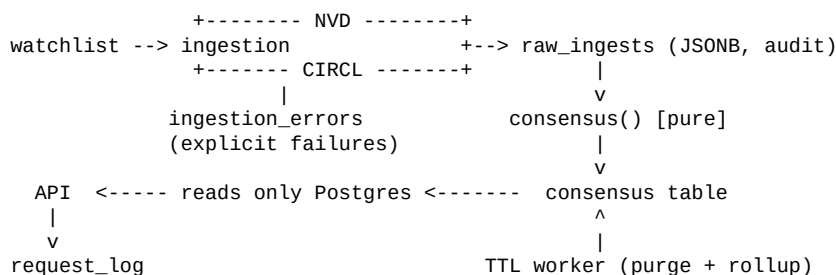
1. Ingestion (the collector). A worker (`app/ingestion.py`) walks a watchlist of 11 well-known CVEs, once or continuously (-- loop, every 15 min). For each one it calls **two independent providers**: NIST’s **NVD** API and Europe’s **CIRCL** CVE service. Every raw response is saved verbatim to the `raw_ingests` table (JSONB) as an audit trail. If a provider times out, is rate-limited (HTTP 429), returns a server error (HTTP 503) or unparseable data, that failure is written explicitly to `ingestion_errors` and the run continues. Old data is never quietly reused and a failed fetch is never presented as a success.

2. Consensus (the judge). A **pure function** (`app/consensus.py`) turns the providers’ answers into one. If both agree on the CVSS score within a tolerance of 1.0, it takes a reliability-weighted average (NVD 0.6 / CIRCL 0.4) and marks the result **verified** with confidence 0.97. If they disagree by more, it flags an outlier disagreement, keeps the more cautious (higher) score and drops confidence to 0.5. If only one provider answered, an answer is still recorded but clearly marked *not two-source verified*. A **quality score** reflects how complete each provider’s record was. The single result is upserted into the consensus table.

3. Storage & lifecycle (the memory). Everything lives in **PostgreSQL** with indexed **JSONB** columns (GIN indexes on payloads and affected products, B-tree on CVE id / source / fetch time). A **TTL worker** (`app/ttl.py`) deletes raw rows older than 3600 s, but first writes one **rollup summary row** per CVE/source recording what was purged, so history is summarised rather than lost. It also marks aged consensus rows as stale.

4. Serving (the counter). A **FastAPI** service (`app/main.py`) exposes GET /v1/security/cve?cve=<ID>. It reads **only from Postgres**, never calling providers during a request, which keeps p95 latency well under 200 ms. It returns the standardised envelope: data plus meta with request id, product id/version, provenance (source, publisher, retrieved_at), freshness (age vs. TTL, stale flag), trust (confidence, quality, verified), license, api latency/rate-limit and warnings. Unknown CVEs get a clean 404 error object; every request is logged to `request_log`.

Data flow: NVD + CIRCL → `raw_ingests` → `consensus()` → consensus table → API → agent, with `ingestion_errors`, `request_log` and TTL rollups guarding integrity and lifecycle.



Testing (16 automated tests, all passing). Unit tests for the pure consensus function (agreement, disagreement, single source, total failure, severity bands, quality). API tests for exact schema shape, verified/unverified, fresh/stale and 404. SLA tests: p95 latency < 200 ms over 100 requests, sustained-polling availability (100 % 200s), and source failover (consensus recomputed and served as degraded when one provider fails).

Section 2 — Use of AI (models and prompts)

Tool: Claude Code (Anthropic's agentic CLI), September 2026.

Models: Claude Opus 4.8 (c laude-opus-4-8) for the initial build; Claude Opus 5 (c laude-opus-5) for the final verification pass.

How it was used. I used Claude Code as a pair-programming assistant working against my written specification. It generated the module layout (app/, migrations/, tests/), the two source adapters, the pure consensus function, the FastAPI serving layer, the migration runner and the pytest suite. It also executed the code in my environment (migration, live ingestion, API, TTL worker, tests) and iterated on failures it observed — most notably rewriting the CIRCL parser after the live API turned out to use the cvelistv5 format with CVSS in an ADP container. I made the design decisions (consensus weighting, tolerance, schema, TTL strategy, watchlist) and approved each step.

Representative prompts.

- “Build a production-style data sourcing, consensus, and serving pipeline for CVE intelligence — Python 3.11+, FastAPI, psycpg2, pytest, Postgres. Ingest from NVD and CIRCL, compute cross-source consensus with confidence/quality scores, store raw + consensus in Postgres JSONB with a 3600 s TTL, and serve the exact standardized JSON envelope. No silent fallbacks; the API reads only from Postgres.” (followed by the full response schema from the brief)
- “The CIRCL source is failing with bad_schema on every CVE — inspect what the CIRCL API actually returns now and fix the parser so real two-source consensus works.”
- “Run everything yourself: migration, ingestion, start the API and query it, run the TTL worker, and run the full test suite — and show me the logs.”
- “See what this assignment is about and make sure nothing is left aside — tell me what remains and run the things for me.” This final pass found that ingestion was one-shot although the brief says *continuously* fetch, so a -- loop mode was added; it also set up PostgreSQL 16 natively and re-ran the whole pipeline and test suite end to end.

What I verified and changed myself.

- Confirmed all 16 tests pass and the served payload matches the required schema exactly (data + meta: provenance, freshness, trust, license, api, warnings).
- Verified real two-source consensus on live data (8/11 CVEs on the final run) and that single-source cases (CIRCL HTTP 429) are recorded in ingestion_errors and marked verified: false. During one run NVD itself returned HTTP 503 for every CVE; every failure was propagated explicitly and the API kept serving clearly-labelled single-source data — no silent fallback.
- Verified the TTL worker purges expired raw rows, writes rollup summaries and marks consensus stale (22 rows purged → 22 rollups, 11 consensus rows stale, checked in the DB).
- Chose the consensus parameters (weights 0.6/0.4, CVSS tolerance 1.0, verified-confidence threshold 0.9) and the watchlist of real CVEs.

Limits of the AI's contribution. The AI did not choose the task, the providers or the trust model; it did not have access to the internet beyond the two public APIs the code calls. All generated code was read and run by me before submission.

Appendix — Results of the final end-to-end run (14 Sep 2026)

Environment: Windows 10, Python 3.13, PostgreSQL 16.10 (native). Commands as in README.md, run in order. Full detail in docs/res.md.

A1. Live ingestion — 11 watchlist CVEs, both providers queried live. Summary: processed 11, consensus_written 11, no_consensus 0, errors 0. Quality 0.9 on two rows: CIRCL carries no product list for those older CVEs.

CVE	Name	Severity	CVSS	Conf.	Quality	Sources	Verified
CVE-2021-44228	Log4Shell	critical	10.0	0.97	1.0	2	yes
CVE-2021-45046	Log4j follow-up	critical	9.0	0.97	1.0	2	yes
CVE-2014-6271	Shellshock	critical	9.8	0.97	0.9	2	yes
CVE-2019-0708	BlueKeep	critical	9.8	0.97	1.0	2	yes
CVE-2022-22965	Spring4Shell	critical	9.8	0.97	1.0	2	yes
CVE-2017-0144	EternalBlue	high	8.8	0.97	1.0	2	yes
CVE-2021-34527	PrintNightmare	high	8.8	0.97	1.0	2	yes
CVE-2023-4863	libwebp	high	8.8	0.97	1.0	2	yes
CVE-2014-0160	Heartbleed	high	7.5	0.97	0.9	2	yes
CVE-2023-44487	HTTP/2 Rapid Reset	high	7.5	0.97	1.0	2	yes
CVE-2020-1472	Zerologon	medium	5.5	0.97	1.0	2	yes

A2. Provider behaviour across the three runs that day. Every failure was written to ingestion_errors and surfaced as a lower-confidence row with an explicit warning — no silent fallback.

Run	NVD	CIRCL	Outcome
1	HTTP 503 on all 11	6 ok, 5 x HTTP 429	6 single-source rows (conf 0.5, unverified), 5 no consensus, 16 errors logged
2	11 ok	8 ok, 3 x HTTP 429	8 verified, 3 single-source, 3 errors logged
3 (final)	11 ok	11 ok	11 verified, 0 errors

A3. API. Envelope matched the Section 3 schema exactly.

Request	Status	Notes
GET /v1/security/cve?cve=CVE-2021-44228	200	cvss 10.0, age 114 s, stale false, verified true, latency 73 ms, warnings []
GET /v1/security/cve?cve=CVE-1999-0001	404	{"error":{"code":"cve_not_found", ...}, "meta":{"...}}

A4. TTL / lifecycle worker. The 1-second run proves the mechanism (one rollup per purged CVE/source group); the normal run purged nothing because all raw rows were younger than the TTL.

Run	TTL	Rollups written	Rows purged	Consensus marked stale
Demonstration	1 s	22	22	11
Final (normal)	3600 s	0	0	0

A5. Test suite — 16 passed, 0 failed, 0 skipped, 2.80 s.

File	Tests	Covers
tests/test_consensus.py	6	agreement, disagreement, single source, total failure, severity bands, quality
tests/test_api.py	6	exact schema shape, verified, 404, stale, fresh, single-source
tests/test_sla.py	4	p95 < 200 ms (60 requests), failover x2, 200-request availability

A6. Database state after the run.

Table	Contents after the final run
raw_ingests	NVD 11 rows (178 kB), CIRCL 11 rows (90 kB), plus 11 NVD:SUMMARY and 11 CIRCL:SUMMARY rollup rows from the TTL demonstration
ingestion_errors	CIRCL / rate_limited x 3 (from run 2)
request_log	534 x HTTP 200 (avg 2 ms, max 101 ms); 3 x HTTP 404 (avg 35 ms)